AP COMPUTER SCIENCE A | STUDENT DIAGNOSTIC HANDOUT

Unit 4 & 6: Deterministic Tracing, Array Iteration & Error Taxonomy

Student Name: _____ **Date:** _____ **Class Period:** _____

Section 1: The Visual Memory Anchor (Mental Model)

> **CORE CONCEPTUAL METAPHOR:** "The Dereference Rule"
> i is the APARTMENT NUMBER (index / memory address).
> numbers[i] is WHO LIVES INSIDE (the actual data stored in that memory cell).
> Never add the apartment numbers when you want to count the residents!

Consider the array declaration: int[] numbers = {3, 7, 2, 9};

Index [0]	Index [1]	Index [2]	Index [3]
Value: 3	Value: 7	Value: 2	Value: 9

Array Properties: numbers.length = 4 | Valid Indices: 0 to (numbers.length - 1) = [0, 1, 2, 3]

Section 2: The Three AP Traversal Traps (Guided Trace Matrix)

BUG 1 | Trap 1: The Accumulator Confusion (Index vs. Dereferenced Value)

```
int[] numbers = {3, 7, 2, 9};
int sum = 0;
for (int i = 0; i < numbers.length; i++) {
    sum += i;  // <-- CRITICAL BUG
}
System.out.println(sum);
```

Pass #	i (Loop Counter)	numbers[i]	Expression Evaluated	sum State
Init	—	—	sum = 0	0
Pass 1	0	3	sum = 0 + 0	0
Pass 2	1	7	sum = 0 + 1	1
Pass 3	2	2	sum = 1 + 2	3
Pass 4	3	9	sum = 3 + 3	6

Output Printed: 6 (AP Distractor Option A: 21) | **Formal Taxonomy:** Logic Error (Index vs. Value confusion) | **Canonical Fix:** sum += numbers[i];

BUG 2 | Trap 2: The Incomplete Traversal (Off-by-One Start Boundary)

```
int[] numbers = {3, 7, 2, 9};
int sum = 0;
for (int i = 1; i < numbers.length; i++) {  // <-- CRITICAL BUG
    sum += numbers[i];
}
System.out.println(sum);
```

Pass #	i (Loop Counter)	numbers[i]	Expression Evaluated	sum State
Init	—	—	sum = 0 (Index 0 skipped!)	0
Pass 1	1	7	sum = 0 + 7	7
Pass 2	2	2	sum = 7 + 2	9
Pass 3	3	9	sum = 9 + 9	18

Output Printed: 18 | **Formal Taxonomy:** Off-by-One Boundary Incompleteness | **AP Penalty:** -1 Point for Incomplete Traversal (ghosted index 0)

BUG 3 | Trap 3: The Runtime Memory Breach (Off-by-One End Boundary)

```
int[] numbers = {3, 7, 2, 9};
int sum = 0;
for (int i = 0; i <= numbers.length; i++) {  // <-- CRITICAL BUG (<=)
    sum += numbers[i];
}
System.out.println(sum);
```

Pass #	i Value	i <= 4 Check	Attempted Array Access	System State / Output
Pass 1	0	0 <= 4 (true)	numbers[0] -> 3	sum = 3
Pass 2	1	1 <= 4 (true)	numbers[1] -> 7	sum = 10
Pass 3	2	2 <= 4 (true)	numbers[2] -> 2	sum = 12
Pass 4	3	3 <= 4 (true)	numbers[3] -> 9	sum = 21
Pass 5	**4**	**4 <= 4 (true)**	**numbers[4] -> DOES NOT EXIST**	**CRASH: ArrayIndexOutOfBounds**

Console Result: ArrayIndexOutOfBoundsException: Index 4 out of bounds for length 4
Crucial AP Exam Insight: Although sum reaches 21 in RAM during Pass 4, System.out.println() NEVER runs. Option '21' is an intentional trap distractor!

Section 3: Differentiated Diagnostic Problem Set

TIER 1 (SUPPORT) | Tier 1: Scaffolded Diagnostic (Sentence Frames & Visual Support)

> **ELL / SCAFFOLD FRAME:** Technical Explanation Frames:
> • "Line ____ contains a [logic error / runtime exception] because it accesses _____ instead of _____."
> • "The loop condition allows i to reach ____, but the maximum valid index is (length - 1) = ____."

Problem 1A: Inspect the following temperature processing routine:

```
double[] dailyTemps = {98.6, 101.2, 99.1};
for (int k = 0; k <= dailyTemps.length; k++) {
    System.out.println("Day " + k + ": " + dailyTemps[k]);
}
```

- **1. Execution Trace:** Program prints Day 0, Day 1, Day 2, and then throws _____ on index _____.
- **2. Code Rectification:** Line 2 loop condition should be changed from k <= dailyTemps.length to _____.

TIER 2 (CORE PROFICIENCY) | Tier 2: Core AP Diagnostic (Algorithmic Tracing)

Problem 2A: The following method is intended to return the minimum value inside array arr.

```
public static int findMin(int[] arr) {
    int min = arr[0];
    for (int i = 1; i <= arr.length - 1; i++) {
        if (arr[i] < min) {
            min = i;  // <-- EXAMINE THIS ASSIGNMENT
        }
    }
    return min;
}

// Client Execution:
int[] data = {14, 8, 22, 5, 19};
System.out.println(findMin(data));
```

- **1. Actual Return Value:** Trace data through findMin(data). What integer is returned to the console? _____ (Expected: 5)
- **2. Error Taxonomy:** Identify the error taxonomy category: _____
- **3. Canonical Fix:** Write the corrected line of code: _____

TIER 3 (ADVANCED EXTENSION) | Tier 3: AP FRQ Challenge (Reverse Traversal & Synthesis)

Problem 3A: A student attempts to copy elements from source into target in reverse order:

```
int[] source = {10, 20, 30, 40};
int[] target = new int[source.length];

// Intended: target becomes {40, 30, 20, 10}
for (int i = source.length; i > 0; i--) {
    target[source.length - i] = source[i];  // <-- EXAMINE ACCESS
}
```

- **1. Execution Crash:** What happens on the very first iteration when i = 4? Explain precisely: _____

- **2. Corrected Header A:** Write Method A (Reverse loop: i starts at source.length - 1 down to 0):
  ```
  for (int i = _____; i >= 0; i--) {
      target[_____] = source[i];
  }
  ```
- **3. Corrected Header B:** Write Method B (Forward loop: i = 0 up to source.length - 1):
  ```
  for (int i = 0; i < source.length; i++) {
      target[i] = source[_____];
  }
  ```

Section 4: AP Exam Free-Response (FRQ) Scoring Rubric Checklist

> **OFFICIAL AP CSA FRQ TRAVERSAL RUBRIC GUIDELINES:** When College Board AP Readers score array traversal questions (FRQ 3/4), full credit requires:
> [+1 pt] Loop Initialization: Initializes loop counter to valid index (e.g., int i = 0).
> [+1 pt] Complete Traversal: Iterates through ALL intended elements without skipping.
> [+1 pt] Bounds Safety: Strictly prevents ArrayIndexOutOfBoundsException (< length).
> [+1 pt] Proper Dereference: Dereferences with arr[i], never accumulating or comparing raw index i.
> Penalty Traps: arr.length() [compile error - .length is a field, not a method] | i <= arr.length [loses 2 pts]

Section 5: Exit Ticket (Individual Formative Assessment)

Consider the method below. Complete the trace table and answer the diagnostic questions independently:

```
public static int mysteryTraversal(int[] vals) {
    int count = 0;
    for (int idx = 0; idx < vals.length; idx += 2) {
        count += vals[idx];
    }
    return count;
}
```

Given: int[] testData = {4, 3, 2, 1, 5};

Pass #	idx Value	vals[idx] Dereferenced	count State
Pass 1	idx = 0	vals[0] = ____	count = 0 + ____ = ____
Pass 2	idx = 2	vals[2] = ____	count = ____ + ____ = ____
Pass 3	idx = 4	vals[4] = ____	count = ____ + ____ = ____

- **Question 1:** Does mysteryTraversal count even numbers, or does it sum elements located at even index positions? Explain: _____
- **Question 2:** What integer value is returned by mysteryTraversal(testData)? _____